

Design docs — per-card + per-skill technical reference

Concise-but-detailed technical design docs for the target-evaluation framework, at two layers:

- `cards/` — one doc per evidence card (the atom): data source → method → emitted categorical → plots → interpretation rules → caveats. This is where the detail lives.
- `skills/` — one thin index per skill: which cards it composes, its verdict ladder, what it does NOT do. Links down to the card docs.

Status: hand-mocked format (2 exemplars)

These are hand-authored mocks to validate the format on real content:

- `cards/expression-distribution.md`
- `skills/tumor-presence.md`

Every field traces to the live contracts (`cards/*.card.yaml`, the two `interpretation-rules/*.yaml`, and each skill's `SKILL.md` + `run.py`) — nothing invented.

Intended end state: generated, never stale

The full set (34 cards + ~12 skills) should be generated from the contracts, not hand-written — hand-written status drifts the moment a card/rule/skill changes. The generator would read `cards/*.card.yaml` (keyed off `schemas/card.schema.json`), the rules files (matched by `when.card_id`), and `SKILL.md` front-matter, reusing the discovery loop in `validators/validate_cards.py` and the `SKILL.md` parser in `skills/_skills_common/composition_schema.py`. Deferred until this format is approved.

Format rules

- Portable GitHub-flavored markdown only — compact tables + fenced ASCII flow. Renders identically in GitHub, VSCode, and `pandoc --from gfm --to docx`.
- No Mermaid (breaks `pandoc→docx`), no HTML.
- Two card generations exist: "modern" cards externalize interpretation to the rules files (read via `when.card_id` grep); 9 "legacy" cards carry inline `interpretation_hints` instead — the card doc's Interpretation section reads whichever applies.